

Guia Prático 3

Controlo do display de 7 segmentos - Parte 2 (GPIO, técnicas de debounce e Timers)

Equipa Docente:

Tiago Gomes

mr.gomes@dei.uminho.pt

Paulo Cardoso

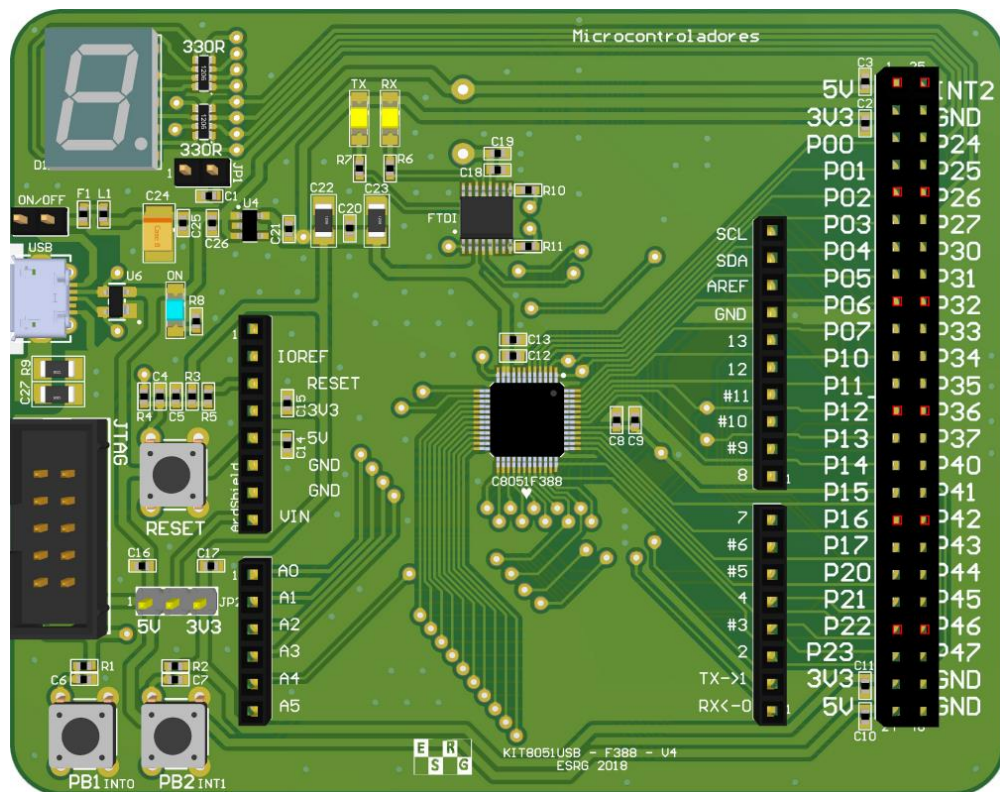
paulo.cardoso@dei.uminho.pt

Embedded Systems Research Group (ESRG)

Departamento de Eletrónica Industrial (DEI) – Universidade Do Minho

1. Objetivos

Controlar os leds do display de 7 segmentos recorrendo aos botões PB1 e PB2 do KIT 8051.
Aprender e aplicar técnicas de debounce.



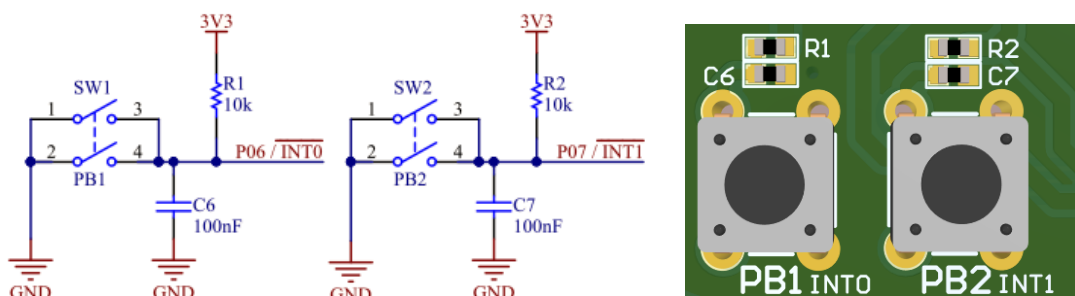
2. Enunciado

Implemente um programa em linguagem C para o microcontrolador C8051F388, que permita interagir com os botões de pressão PB1 e PB2 e escreva informação no display de sete segmentos. Mais concretamente, o programa deve incrementar (PB1) / decrementar (PB2) um valor em memória ('0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F') apresentando-o no display. Quando o valor mínimo/máximo for atingido, o valor a apresentar no display deve ser mantido. O display inicia com o valor zero ('0'). Deve ser também verificado o problema de bounce nas teclas PB1 e PB2, e aplicada uma técnica em software para o resolver.

3. Silicon Labs IDE / Keil

PARTE 1

- A. Crie um novo projeto no IDE como aprendeu nos guias anteriores, e configure-o para utilizar o microcontrolador (MCU) C8051F388, presente no KIT8051 que lhe foi fornecido.
- B. Comece por perceber as ligações físicas (verificar esquemático da placa) entre o MCU e as teclas, e entre o MCU e o display 7 segmentos. Entender que valores deve o MCU armazenar para que a informação possa ser apresentada no display.

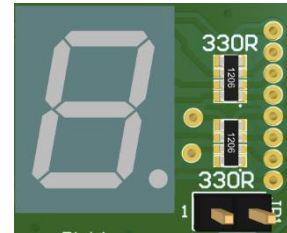
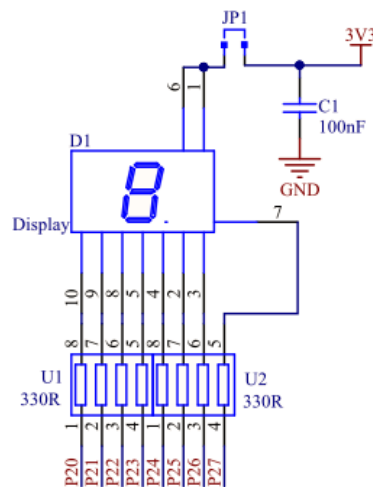


Q1: Para se poder ler o valor dos botões PB1 (P6.0) e PB2 (P0.7), que configurações deve ter o Porto/Pinos do MCU?

Q2: O botão de pressão (SW na imagem) fecha um circuito quando premido. Por default, tecla com circuito em aberto, qual o valor de tensão e lógico presentes nos respetivos pinos P06 e P07?

Q3: Quais as vantagens/desvantagens do circuito demonstrado?

Verifique agora as ligações físicas entre o display de 7 segmentos e o MCU.



Q4: Que acontece se for fechado o circuito do Pino P2.0 com GND? E VCC? Justifique a resposta e verifique na placa, criando um pequeno programa que atue apenas no pino P2.0.

Q5: Para mostrar o dígito zero no display, que segmentos (de A a G) devem estar ligados?

Q6: Estando os segmentos do display de 7 segmentos mapeados fisicamente em P2, que valor deve ser colocado em todo o P2? E para os restantes valores (1 a F)?

Dica1: Para o valor zero, deve ser colocado em P2 o valor 0xC0.

Dica2: Perceber as configurações necessárias no MCU para que se possa utilizar os periféricos necessários (GPIO no MCU e restante hardware no KIT8051), recorrer ao datasheet e à ferramenta configwizard2.

- C. Desenvolva um algoritmo que descreva o enunciado proposto e implemente o programa recorrendo à linguagem C.

Proposta (incompleta) de implementação:

```
#include <REG51F380.H>

sbit PB1 = P0^6; //PB1 is connected to Port 0, pin 6
sbit PB2 = P0^7; //PB2 is connected to Port 0, pin 7

/* force this array to be stored in the program memory.
 * Program (CODE) memory is read-only! */
code const char seg[] = {0xC0}; //values to put in the display
code const int seg_size = sizeof(seg)/sizeof(char);

/*****
void system_init(void){
    PCA0MD    = 0x00;
    FLSEL     = 0x90;
    CLKSEL    = 0x03;
    XBR1      = 0x40;
}
```

```

/*****
 *      main() loop      *
 *****/
void main (void){

// chose your vars...
int index = 0; // this var controls the index value of the array element being displayed
/* read push buttons */
bit pbl, pb2; // vars to store the push buttons values

system_init();

// in debug, test and validate here all the values
while (index <= seg_size){
    P2 = seg[index];
}

while(1){
    pbl = PB1; //read the actual value of PB1
    pb2 = PB2; //read the actual value of PB2

//detect negedge
if(pbl){ //check other conditions
    // do something
}

if(pb2){ //check other conditions
    // do something
}
// update display, accordingly
P2 = seg[index];
}
}
/*****/

```

Sugestões:

- i. Guardar num array os valores a colocar em P2, de acordo com o que calculou nas questões Q5 e Q6.
- ii. Testar, em modo debug “passo a passo”, se os valores são atualizados corretamente no display de acordo com o que calculou.

```

// in debug, test and validate here all the values
while (index <= seg_size){
    P2 = seg[index];
}

```

- iii. O algoritmo deve seguir uma lógica que não bloqueie a sua execução. Usar variáveis para guardar o estado atual da tecla, verificar com o estado anterior, e comparar se houve alterações no mesmo. Variáveis pb1 e pb2 têm o valor atual da tecla, pb1_prev e pb2_prev guardam o valor anterior.

```

/* read push buttons */
bit pbl, pb2, pbl_prev = 1, pb2_prev = 1;

// Detect edge change for PB1, from HIGH to LOW
if ((pbl_prev == 1) && (pbl == 0)){

    // do something for this button
}

// save current button state
pbl_prev = pbl;

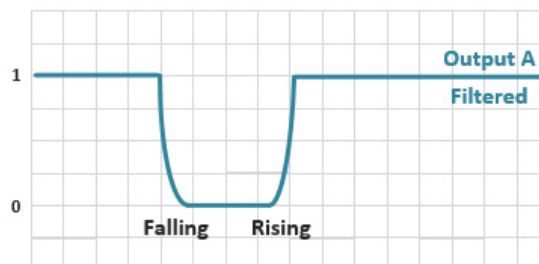
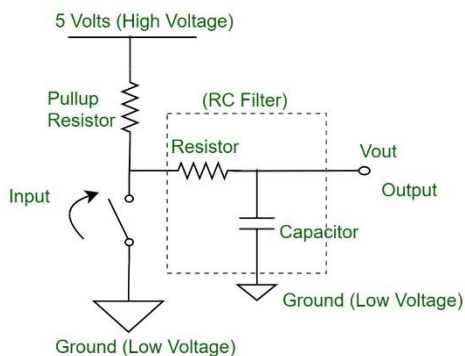
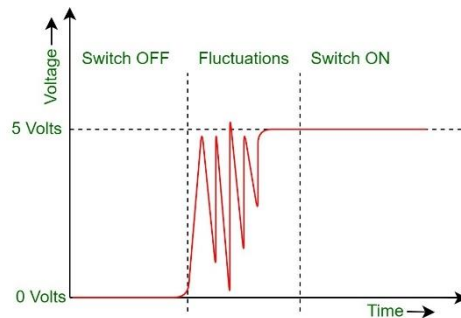
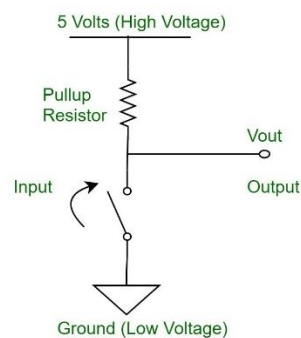
```

Elementos a entregar – 11/10/2024 (Ficheiro zip com):

- Ficheiro txt com as respostas às questões colocadas Q1 a Q6.
- Lista/tabela dos valores a armazenar na memória, correspondendo aos dígitos a mostrar no display;
- Print do keil com os valores a serem atualizados no P2 e foto/imagem da placa com esta atualização;
- Algoritmo com a solução proposta (pseudo-código, fluxogramas);
- Projeto keil (PASTA TODA, APENAS COM ESTE PROJETO), com o código e outros *source files* que achar relevantes. Submissões fora de prazo e com apenas ficheiro .c, e outros ficheiros dentro da pasta que não pertençam a este projeto não serão considerados.

PARTE 2

Estudar o problema do bounce encontrado nos botões de pressão/mecânicos e as soluções que existem para o resolver.



- D. Verificar se existe o problema de bounce nas teclas com a ajuda do osciloscópio presente no laboratório e estude um algoritmo que o resolva, devendo ser implementado também no programa desenvolvido. Mesmo que o problema não seja frequentemente, graças ao filtro RC já presente no circuito, **deve implementar na mesma o algoritmo em software**. Caso queira testar com um circuito sem o filtro RC, pode adicionar um push button externo apenas com a resistência de PULL-UP.

Propostas de implementação de um algoritmo de debounce

- E. Implementar uma função de debounce. Mudar a leitura das variáveis pb1 e pb2. Em vez de lerem dos botões físicos passam a ser atualizados com os valores calculados nas funções de debounce. Isto permite reutilizar todo o código feito até ao momento. Testar e validar a estratégia no modo debug antes de implementar.

Usando a estratégia de Counter-Based Debounce

Criar uma variável para armazenar o resultado do debounce:

bit pb1_d;

```
void soft_delay_ms(unsigned int ms){
    unsigned int i, j;

    for (i = 0; i < ms; i++){
        for(j = 0; j < 65535; j++){
            // Adjust this loop. Toggle a pin to measure the period of this delay
        }
    }
}

void simple_debounce_pb1(){
    unsigned char count = 0;
    bit PBl_state = PBl; // Store the initial state of the button

    // Check if the button is pressed (active-low)
    if (PBl == 0) { // Button press detected
        // Perform x consecutive checks with 10ms delay
        while (count < 5){
            soft_delay_ms(10);
            if (PBl == 0) {
                count++; // Count stable presses
            } else {
                pbl_d = PBl_state; // Return to initial state on bounce
                return; // Button bounced, return
            }
        }
        pbl_d = 0; // Update global variable to indicate stable press (0)
    } else { // Check for button release
        count = 0; // Reset count for release check
        // Perform 10 consecutive checks for button release
        while (count < 5) {
            soft_delay_ms(10);
            if (PBl == 1) { // Button is released
                count++; // Count stable releases
            } else {
                pbl_d = PBl_state; // Return to initial state on bounce
                return; // Button bounced, return
            }
        }
        pbl_d = 1; // Update global variable to indicate stable release (1)
    }
}
```

Na solução com um hardware timer, a atualização das variáveis de debounce podem ser feitas na ISR. Será feito no próximo guia.

Usando a estratégia de Software Filtering with Shift Registers

```
// Global variable to store the stable state of the button
bit pbl_d = 1; // Initialize to 1 (button released)
static uint8_t state = 0; // 8-bit shift register for button states
static unsigned char stable_count = 0; // Counter for stable readings

#define STABLE_THRESHOLD 5 // Number of stable readings
#define CHECK_INTERVAL 10 // Time to wait between checks in milliseconds

void shift_debounce_pbl(void) {

    // Update the state with the current button reading
    state = (state << 1) | (PB1 == 0 ? 1 : 0); // Shift left and add current state

    // Check for stable pressed state (0x00) and stable released state (0xFF)
    if (state == 0x00) {
        stable_count++; // Increment stable count for pressed state
    } else if (state == 0xFF) {
        stable_count++; // Increment stable count for released state
    } else {
        stable_count = 0; // Reset stable count if state is not stable
    }

    // Update pbl only after reaching the stable threshold
    if (stable_count >= STABLE_THRESHOLD) {
        pbl_d = (state == 0x00) ? 0 : 1; // Update pbl based on stable state
    }
}
```

This on the main loop...

```
// different approaches to upload pbl...
// comment what you don't need

//without debounce
pbl = PB1;

// with simple_delay debounce
// change the var pbl to read the debounce var instead of physical button

simple_debounce_pbl();
pbl = pbl_d;

// with shift register debounce technique
// Call debounce function in a loop with delay to check state every 10ms

for (i = 0; i < STABLE_THRESHOLD; i++) {
    shift_debounce_pbl(); // Call debounce function to update pbl_d
    soft_delay_ms(CHECK_INTERVAL); // Wait for 10 ms between checks
}
pbl = pbl_d;
```

Elementos a entregar – 18/10/2024 (Ficheiro zip com):

- Ficheiro **zip** com **TODO** o projeto e com as linhas de código/funções/funcionalidades devidamente comentadas. Fluxogramas/diagramas com o algoritmo final. Bonificação para quem implementar o melhor projeto com debounce por software. Submissões fora de prazo não serão consideradas.

PARTE 3

Estudar os Timers que existem no MCU. Atualizar e melhorar a técnica de debounce usada na PARTE 2 com a ajuda de um Timer.

- F. Faça download do ficheiro 1-BlinkLed - timer2-polling.zip. Deve extrair o projeto e executar no kit que tem disponível.

```
23  /*****
24  * timer2_init_auto function
25  *****/
26  void timer2_init_auto(int reload) {
27
28      // Timer 2 control register
29      TMR2CN = 0;
30      // Use system clock
31      CKCON &= ~(1 << 5);
32
33      // set load values
34      TMR2H = (reload) >> 8;
35      TMR2L = (reload);
36
37      // set auto reload values for autoreload mode
38      TMR2RLH = (reload) >> 8;
39      TMR2RLL = (reload);
40  }
```

Q7: Explique a função de inicialização do timer 2, timer2_init_auto (int reload);, ao nível dos registos e dos valores lá colocados, modo de operação, etc.

Q8: Além do modo identificado em cima, que outros modos de operação suporta o timer 2?

Q9: Qual a configuração do timer 2 para funcionar como dois timers independentes de 8 bits? Escreva a função timer2_init_8bit_auto(int reloadt1, int reloadt2);

Q10: Como monitorizar o overflow de cada um?

Q11: Que outros clock sources o timer 2 suporta?

Q12: Como mudar a frequência do timer 2 para 48 MHz? quais as vantagens/desvantagens?

Q13: Com clock source de 48MHz quanto tempo dura uma contagem completa do timer em 16 bit auto reload?

Q14: Se quisermos usar um clock source com uma frequência de SYSCLK / 48, que outros temporizadores podemos usar?

Q15: Qual seria a duração da contagem completa se configurado em 16 bits?

Q16: Após configurado o timer, o exemplo apresenta o seguinte código na função main(). Explique o que fazem as restantes linhas de código incluindo o timer2_init_auto(-4000);.


```

49 // Set Timer 2 to overflow every 10ms
50 // (with a 4 MHz clock, 10 ms = 40000 ticks)
51 timer2_init_auto(-40000);
52
53 //Set by hardware when the Timer 2 high byte overflows from 0xFF to 0x00. In 16 bit
54 //mode, this will occur when Timer 2 overflows from 0xFFFF to 0x0000.
55 TF2H = 0;
56 // Enable Flag Timer 2 Overflow
57 ET2 = 1;
58
59 // Timer 2 Run Control. Timer 2 is enabled by setting this bit to 1.
60 TR2 = 1;
61
62 while(1){
63
64     while(!TF2H);
65     TF2H = 0;
66
67     // seg_dp ^=1;
68
69     if(--blink == 0){
70         seg_dp ^= 1;
71         blink = 100;
72     }
73 }
74 }

```

Q16: Que modificações faria para fazer o toggle do led a cada 555ms? Altere o exemplo fornecido.

G. Faça download do ficheiro 2-BlinkLed - timer2-interrupt.zip e inclua as alterações feitas anteriormente. Teste o funcionamento e valide com o osciloscópio. Devem ser submetidos prints/foto do que apareceu no osciloscópio.

H. No slides verificou que se podiam usar software timers para explorar um hardware timer, permitindo assim maior otimização de recursos.

Q17: Que outras vantagens tem a utilização de software timers?

I. Escolha umas das implementações fornecidas com software timers. Recomenda-se a utilização do projeto com funções de callback para simplificar a sua utilização com outros projetos que já tenha concluído.

Q17: Modifique ou apenas configure o código para que possam ser usados 2 software timers, e cada função de callback possa fazer toggle a um pino de GPIO: P1.2 a cada 133 ms, P1.3 a cada 215ms. Confirme no osciloscópio e registre o resultado.

J. Utilizando a implementação do software timer que escolheu, atualize a rotina de debounce que desenvolveu no projeto/guia anterior. Um software timer deve executar uma rotina de callback a cada 10 ms. Nessa rotina deve implementar o algoritmo que escolheu, em que a variável de debounce, e.g., pb1_d e pb2_d são atualizadas periodicamente.

Elementos a entregar – 25/10/2024 (Ficheiro zip com):

- Ficheiro **zip** com **TODO** o projeto final do guia anterior, agora com o algoritmo de debounce e com o software timer (todas as linhas de código/funções/funcionalidades que achar convenientes devem estar devidamente comentadas). Deve entregar um ficheiro txt com as respostas às perguntas que foram colocadas e incluir os prints/fotos sempre que necessário.
- **Submissões fora de prazo não são consideradas.**